

Лабораторная работа №6

Решение моделей в непрерывном и дискретном времени

Доберштейн А. С.

Российский университет дружбы народов, Москва, Россия

Информация

- Доберштейн Алина Сергеевна
- НФИбд-02-22
- Российский университет дружбы народов

Цель работы

Основной целью работы является освоение специализированных пакетов для решения задач в непрерывном и дискретном времени.

```
[1]:  
  
# Модель экспоненциального роста  
# подключаем необходимые пакеты:  
import Pkg  
Pkg.add("DifferentialEquations")  
using DifferentialEquations  
  
[32m[1m Resolving[22m[39m package versions... •••
```

Рис. 1: Решение обыкновенных дифференциальных уравнений

```
[2]:  
  
using DifferentialEquations  
# задаём описание модели с начальными условиями:  
a = 0.98  
f(u,p,t) = a*u  
u0 = 1.0  
# задаём интервал времени:  
tspan = (0.0,1.0)  
# решение:  
prob = ODEProblem(f,u0,tspan)  
sol = solve(prob)
```

```
[2]:  
  
retcode: Success  
Interpolation: 3rd order Hermite  
t: 5-element Vector{Float64}:  
 0.0  
 0.10042494449239292  
 0.3521860297865888  
 0.6934436122197829  
 1.0  
u: 5-element Vector{Float64}:  
 1.0  
 1.1034222047865465  
 1.412190871348492  
 1.9730384457359196  
 2.664456142481427
```

Рис. 2: Решение обыкновенных дифференциальных уравнений

```
# подключаем необходимые пакеты:
```

```
Pkg.add("Plots")
```

```
using Plots
```

```
[32m[1m Resolving[22m[39m package versions... •••
```

```
[3]:
```

```
using Plots
```

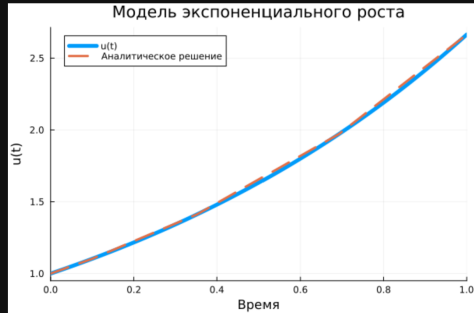
```
# строим графики:
```

```
plot(sol, linewidth=5, title="Модель экспоненциального роста",
```

```
      xaxis="Время", yaxis="u(t)", label="u(t)")
```

```
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, label="Аналитическое решение")
```

```
[3]:
```



```
# задаём точность решения:  
sol = solve(prob, abstol=1e-8, reltol=1e-8)  
println(sol)  
# строим график:  
plot(sol, lw=2, color="black", title="Модель экспоненциального роста",  
      xaxis="Время", yaxis="u(t)", label="Численное решение")  
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, color="red", label="Аналитическое  
решение")
```

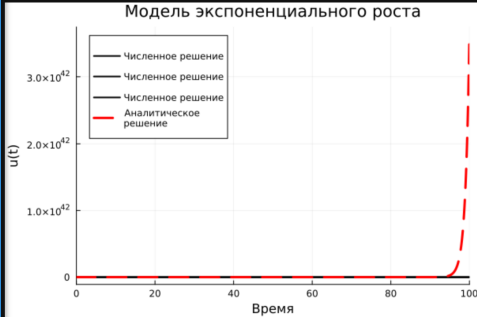


Рис. 4: Решение обыкновенных дифференциальных уравнений

Решение ОДУ

```
#Система Лоренца
# подключаем необходимые пакеты:
using DifferentialEquations, Plots;
# задаём описание модели:
function lorenz!(du,u,p,t)
     $\sigma, \rho, \beta = p$ 
    du[1] =  $\sigma \cdot (u[2] - u[1])$ 
    du[2] =  $u[1] \cdot (\rho - u[3]) - u[2]$ 
    du[3] =  $u[1] \cdot u[2] - \beta \cdot u[3]$ 
end
# задаём начальное условие:
u0 = [1.0, 0.0, 0.0]
# задаём значения параметров:
p = (10, 28, 8/3)
# задаём интервал времени:
tspan = (0.0, 100.0)
# решение:
prob = ODEProblem(lorenz!, u0, tspan, p)
sol = solve(prob)
```

[6]:

```
retcode: Success
Interpolation: 3rd order Hermite
t: 1303-element Vector{Float64}:
 0.0
 3.5678604836301404e-5
 0.0003924646531993154
 0.0032623945564751056
 0.009058054066400603
 0.016956441391185063
 0.02768991931434415
 0.04185629724071979
 0.060240340738233165
 0.08368532068705861
 0.11336487743821547
 0.14862168348543542
 0.1870396396499564
```

```
using Plots
# строим график:
plot(sol, vars=(1,2,3), lw=0.5, title="Аттрактор Лоренца",
      xaxis="x", yaxis="y", zaxis="z", legend=false)

[ Warning: To maintain consistency with solution indexing, keyword argument vars will be r
emoved in a future version. Please use keyword argument idxs instead.
 caller = ip:0x0
 ]
```

[7]:



Рис. 6: Решение обыкновенных дифференциальных уравнений

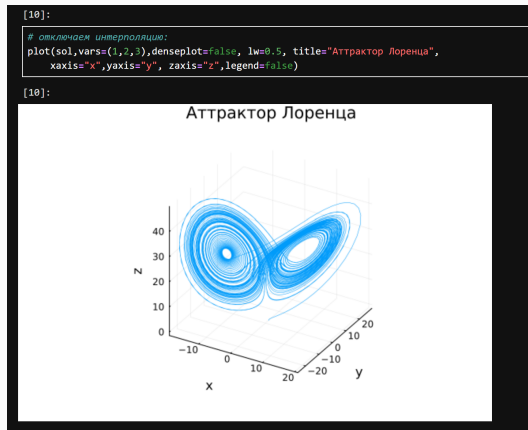


Рис. 7: Решение обыкновенных дифференциальных уравнений

Модель Лотки–Вольтерры

[12]:

```
#Модель Лотки–Вольтерры
# подключаем необходимые пакеты:
import Pkg
Pkg.add("ParameterizedFunctions")
```

⌕[32m⌕[1m Resolving⌕[22m⌕[39m package versions... ●●●

[13]:

```
using ParameterizedFunctions, DifferentialEquations, Plots;
# задаём описание модели:
lv! = @ode_def LotkaVolterra begin
    dx = a*x - b*x*y
    dy = -c*y + d*x*y
end a b c d
# задаём начальное условие:
u0 = [1.0, 1.0]
# задаём значения параметров:
p = (1.5, 1.0, 3.0, 1.0)
# задаём интервал времени:
tspan = (0.0, 10.0)

# решение:
prob = ODEProblem(lv!, u0, tspan, p)
sol = solve(prob)
plot(sol, label = ["Жертвы" "Хищники"], color="black", ls=[:solid
:dash], title="Модель Лотки - Вольтерры",
xaxis="Время", yaxis="Размер популяции")
```

Рис. 8: Модель Лотки–Вольтерры

Модель Лотки–Вольтерры

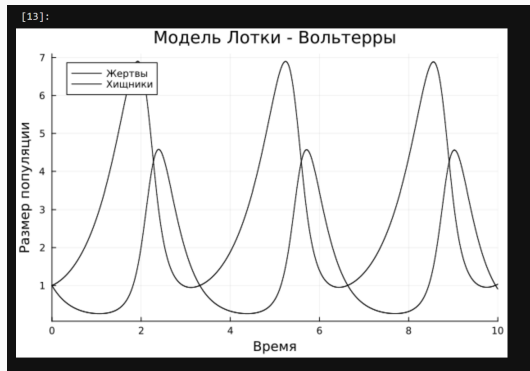


Рис. 9: Модель Лотки–Вольтерры

Модель Лотки–Вольтерры

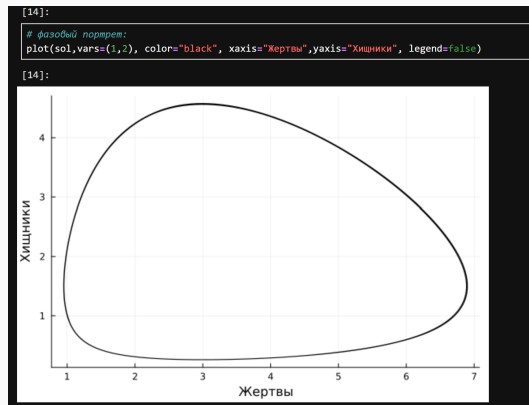


Рис. 10: Модель Лотки–Вольтерры

Задания для самостоятельного выполнения

1. Реализовать и проанализировать модель роста численности изолированной популяции (модель Мальтуса):

$$\dot{x} = ax, \quad a = b - c.$$

где $x(t)$ — численность изолированной популяции в момент времени t , a — коэффициент роста популяции, b — коэффициент рождаемости, c — коэффициент смертности. Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией).

[15]:

```
# 1

using DifferentialEquations, Plots;

b = 0.08
c = 0.06
a = b - c

f(u,p,t) = a*u
u0 = 1

tspan = (0.0, 100.0)

prob = ODEProblem(f, u0, tspan)
sol = solve(prob)

plot(sol, linewidth=1, title="Модель Мальтуса", xaxis="Время", yaxis="Численность", label=
plot!(sol.t, t->1.0*exp(a*t), lw=2, ls=:dash, label="Аналитическое решение")
```

Рис. 11: Задание для самостоятельного выполнения №1

Задания для самостоятельного выполнения

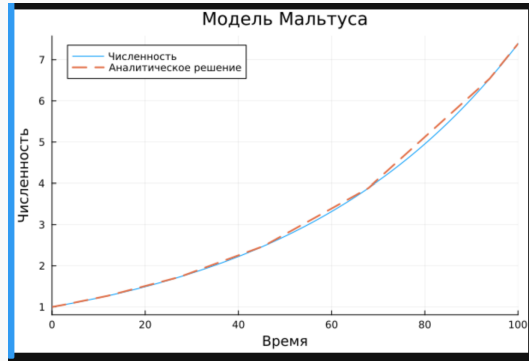


Рис. 12: Задание для самостоятельного выполнения №1

Задания для самостоятельного выполнения

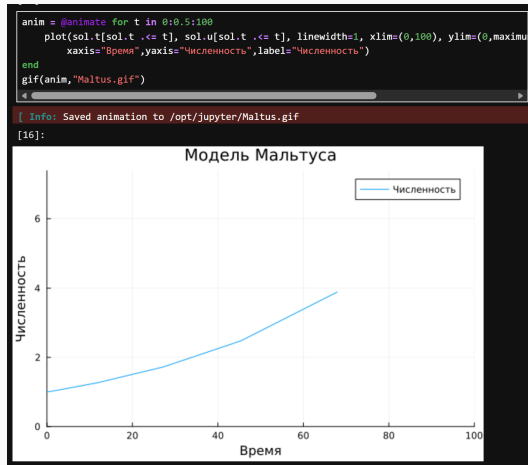


Рис. 13: Задание для самостоятельного выполнения №1

Задания для самостоятельного выполнения

2. Реализовать и проанализировать логистическую модель роста популяции, заданную уравнением:

$$\dot{x} = rx \left(1 - \frac{x}{k}\right), \quad r > 0, \quad k > 0,$$

r — коэффициент роста популяции, k — потенциальная ёмкость экологической системы (предельное значение численности популяции). Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией).

[17]:

```
# 2

r = 0.2
k = 100

f(u,p,t) = r*u*(1-(u/k))

u0 = 1
tspan = (1,100)

prob = ODEProblem(f,u0,tspan)
sol = solve(prob)

plot(sol, linewidth=1, title="Логистическая модель",
      xaxis="Время", yaxis="Численность", label="Численность")
plot!(sol.t, t->(100/(-.00827001*exp(t/5)-1))+100, lw=2, ls=:dash, label="Аналитическое решен
```

Рис. 14: Задание для самостоятельного выполнения №2

Задания для самостоятельного выполнения

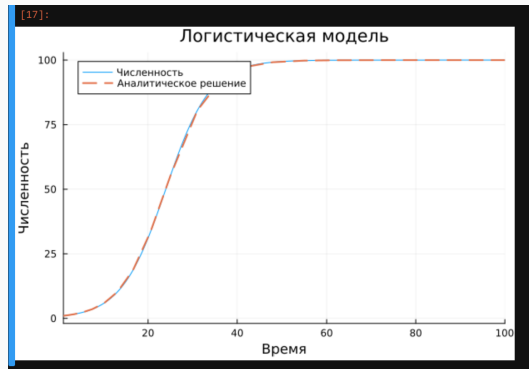


Рис. 15: Задание для самостоятельного выполнения №2

Задания для самостоятельного выполнения

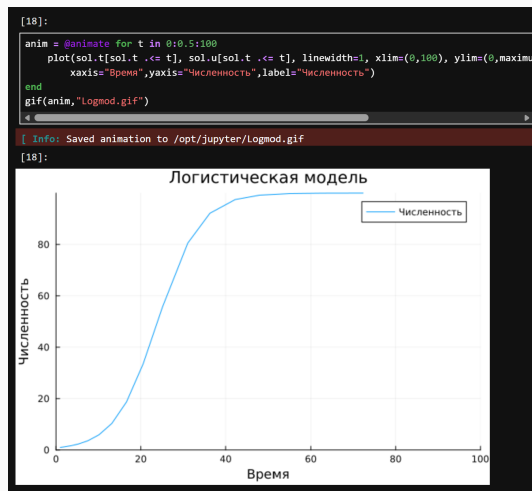


Рис. 16: Задание для самостоятельного выполнения №2

Задания для самостоятельного выполнения

3. Реализовать и проанализировать модель эпидемии Кермака–Маккендрика (SIR-модель):

$$\begin{cases} \dot{s} = -\beta i s, \\ \dot{i} = \beta i s - \nu i, \\ \dot{r} = \nu i, \end{cases}$$

где $s(t)$ — численность восприимчивых к болезни индивидов в момент времени t , $i(t)$ — численность инфицированных индивидов в момент времени t , $r(t)$ — численность переболевших индивидов в момент времени t , β — коэффициент интенсивности контактов индивидов с последующим инфицированием, ν — коэффициент интенсивности выздоровления инфицированных индивидов. Численность популяции считается постоянной, т.е. $\dot{s} + \dot{i} + \dot{r} = 0$. Начальные данные и параметры задать

•[19]:

```
using DifferentialEquations, Plots;
# задаём описание модели:
function SIR(du,u,p,t)
    b, v = p
    du[1] = -b*u[2]*u[1]
    du[2] = b*u[2]*u[1]-v*u[2]
    du[3] = v*u[2]
end
# задаём начальные условия:
u0 = [4.0, 2.0, 2.0]
# задаём значения параметров:
p = (0.3,0.32)
# задаём интервал времени:
tspan = (0.0,10.0)
# решение:
prob = ODEProblem(SIR,u0,tspan,p)
sol = solve(prob)

# строим графики:
plot(sol, linewidth=1,title="SIR-модель",
      xaxis="Время",yaxis="Численность",label=["s" "i" "r"])
```

Рис. 17: Задание для самостоятельного выполнения №3

Задания для самостоятельного выполнения

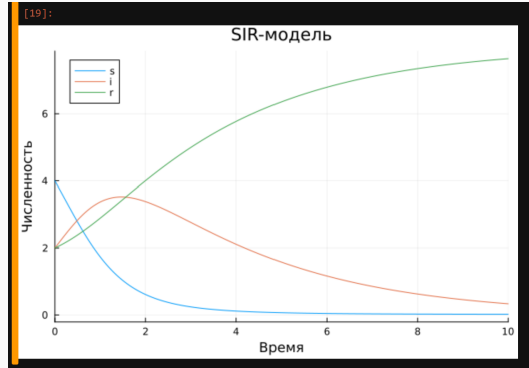


Рис. 18: Задание для самостоятельного выполнения №3

Задания для самостоятельного выполнения

4. Как расширение модели SIR (Susceptible-Infected-Removed) по результатам эпидемии испанки была предложена модель SEIR (Susceptible-Exposed-Infected-Removed):

$$\begin{cases} \dot{s}(t) = -\frac{\beta}{N}s(t)i(t), \\ \dot{e}(t) = \frac{\beta}{N}s(t)i(t) - \delta e(t), \\ \dot{i}(t) = \delta e(t) - \gamma i(t), \\ \dot{r}(t) = \gamma i(t). \end{cases}$$

Размер популяции сохраняется:

$$s(t) + e(t) + i(t) + r(t) = N.$$

•[20]:

```
#b - скорость передачи инфекции (b)
#d - скорость перехода из e в i (d)
#gamma - скорость выздоровления (g)
using DifferentialEquations, Plots;
# задаём описание модели:
function SEIR(du,u,p,t)
    b, d, g = p
    du[1] = -(b/N)*u[1]*u[3]
    du[2] = (b/N)*u[1]*u[3]-d*u[2]
    du[3] = d*u[2] - g*u[3]
    du[4] = g*u[3]
end
# задаём начальное условие:
u0 = [2.0, 3.0, 2.0, 2.0]
# задаём значения параметров:
N = u0[1]+u0[2]+u0[3]+u0[4]
p = (0.3, 0.2, 0.32)
# задаём интервал времени:
tspan = (0.0,10.0)
# решение:
prob = ODEProblem(SEIR,u0,tspan,p)
sol = solve(prob)
```

Задания для самостоятельного выполнения



Рис. 20: Задание для самостоятельного выполнения №4

Задания для самостоятельного выполнения

5. Для дискретной модели Лотки–Вольтерры:

$$\begin{cases} X_1(t+1) = aX_1(t)(1 - X_1(t)) - X_1(t)X_2(t), \\ X_2(t+1) = -cX_2(t) + dX_1(t)X_2(t). \end{cases}$$

с начальными данными $a = 2, c = 1, d = 5$ найдите точку равновесия. Получите и сравните аналитическое и численное решения. Численное решение изобразите на фазовом портрете.

[21]:

```
#5
a, c, d = 2, 1, 5
t_max = 100
x1_0 = 0.5
x2_0 = 0.3
X1 = zeros(t_max)
X2 = zeros(t_max)
X1[1] = x1_0
X2[1] = x2_0
for t in 1:t_max-1
    X1[t+1] = a * X1[t] * (1 - X1[t]) - X1[t] * X2[t]
    X2[t+1] = -c * X2[t] + d * X1[t] * X2[t]
end
plot(1:t_max, X1, label="Жерты", lw=2)
plot(1:t_max, X2, label="Хищники", xlabel="Время", ylabel="Численность", title="Численное")
```

Рис. 21: Задание для самостоятельного выполнения №5

Задания для самостоятельного выполнения

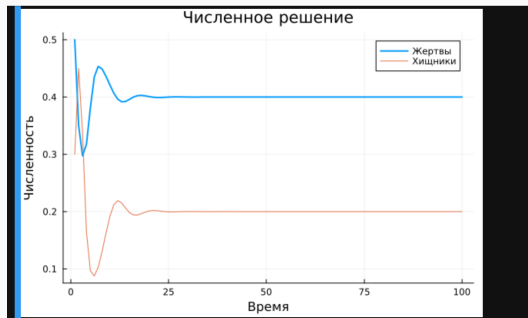


Рис. 22: Задание для самостоятельного выполнения №5

Задания для самостоятельного выполнения

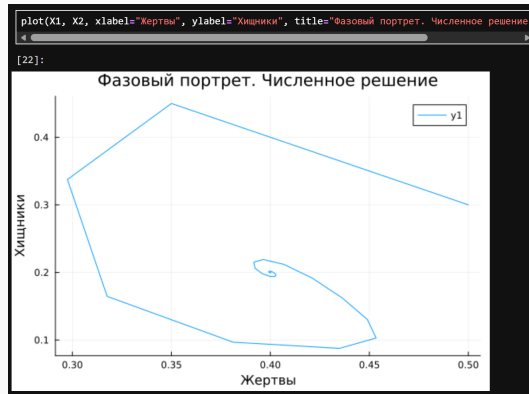


Рис. 23: Задание для самостоятельного выполнения №5

Задания для самостоятельного выполнения

6. Реализовать на языке Julia модель отбора на основе конкурентных отношений:

$$\begin{cases} \dot{x} = \alpha x - \beta xy, \\ \dot{y} = \alpha y - \beta xy. \end{cases}$$

Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

[23]:

```
#6
using DifferentialEquations, Plots;
#a - коэффициент естественного прироста
#b - коэффициент конкурентного взаимодействия
# задаём описание модели:
function f(du,u,p,t)
    a, b = p
    du[1] = a*u[1]-b*u[1]*u[2]
    du[2] = a*u[2]-b*u[1]*u[2]
end
# задаём начальное условие:
u0 = [10.0, 15.0]
# задаём значения параметров:
p = (0.8, 1)
# задаём интервал времени:
tspan = (0.0,100.0)
# решение:
prob = ODEProblem(f,u0,tspan,p)
sol = solve(prob)

# строим графики:
plot(sol, linewidth=1,title="Конкурентные отношения",
      xaxis="Время",yaxis="Численность")
```

Рис. 24: Задание для самостоятельного выполнения №6

Задания для самостоятельного выполнения

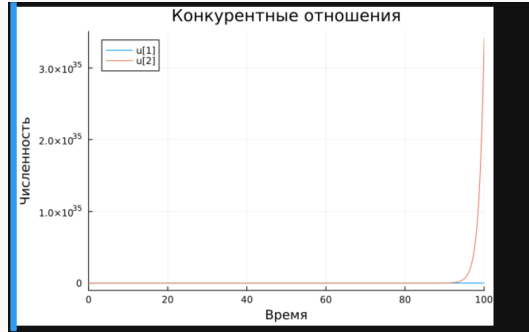


Рис. 25: Задание для самостоятельного выполнения №6

Задания для самостоятельного выполнения

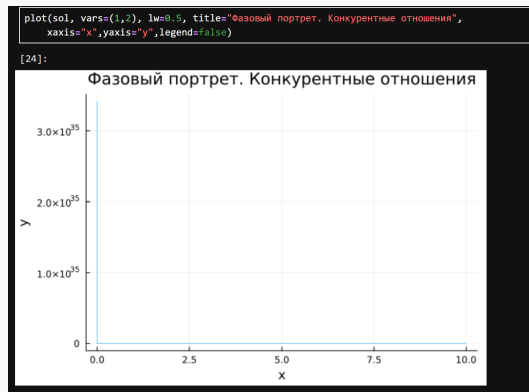


Рис. 26: Задание для самостоятельного выполнения №6

Задания для самостоятельного выполнения

7. Реализовать на языке Julia модель консервативного гармонического осциллятора

$$\ddot{x} + \omega_0^2 x = 0, \quad x(t_0) = x_0, \quad \dot{x}(t_0) = y_0,$$

где ω_0 — циклическая частота. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

[25]:

```
#7
function f(du,u,p,t)
    o, g = p
    du[1] = u[2] #dx
    du[2] = -2*g*u[2]-o^2*u[1] #dy x''
end
# задаём начальное условие:
u0 = [1.0, 2.0]
# задаём значения параметров:
p = (3, 0)
# задаём интервал времени:
tspan = (0.0, 10)
# решение:
prob = ODEProblem(f,u0,tspan,p)
sol = solve(prob)

# строим графики:
plot(sol, linewidth=1,title="Консервативный гармонический осциллятор")
```

Рис. 27: Задание для самостоятельного выполнения №7

Задания для самостоятельного выполнения

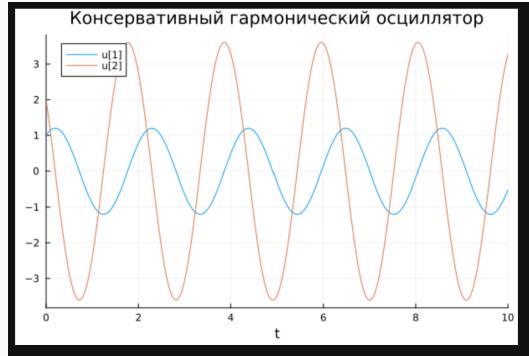


Рис. 28: Задание для самостоятельного выполнения №7

Задания для самостоятельного выполнения

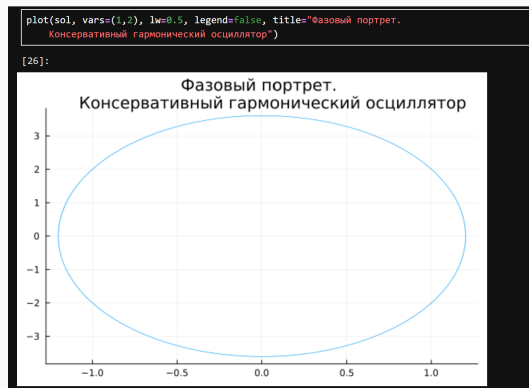


Рис. 29: Задание для самостоятельного выполнения №7

Задания для самостоятельного выполнения

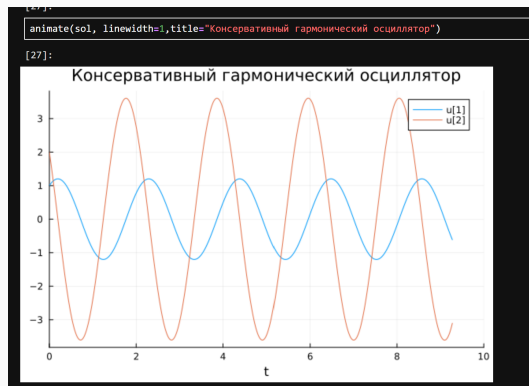


Рис. 30: Задание для самостоятельного выполнения №7

Задания для самостоятельного выполнения

8. Реализовать на языке Julia модель свободных колебаний гармонического осциллятора

$$\ddot{x} + 2\gamma\dot{x} + \omega_0^2 x = 0, \quad x(t_0) = x_0, \quad \dot{x}(t_0) = y_0,$$

где ω_0 — циклическая частота, γ — параметр, характеризующий потери энергии. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

[28]:

```
#8
function f(du,u,p,t)
    o, g = p
    du[1] = u[2] #dx
    du[2] = -2*g*u[2]-o^2*u[1] #dy x''
end
# задаём начальное условие:
u0 = [1.0, 2.0]
# задаём значения параметров:
p = (3, 0.1)
# задаём интервал времени:
tspan = (0.0, 10)
# решение:
prob = ODEProblem(f,u0,tspan,p)
sol = solve(prob)

# строим графики:
plot(sol, linewidth=1,title="Модель свободных колебаний
    гармонического осциллятора")
```

Рис. 31: Задание для самостоятельного выполнения №8

Задания для самостоятельного выполнения

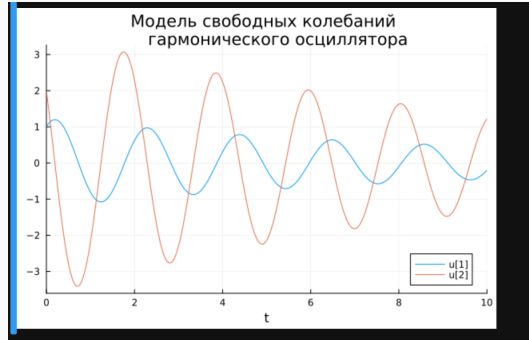


Рис. 32: Задание для самостоятельного выполнения №8

Задания для самостоятельного выполнения

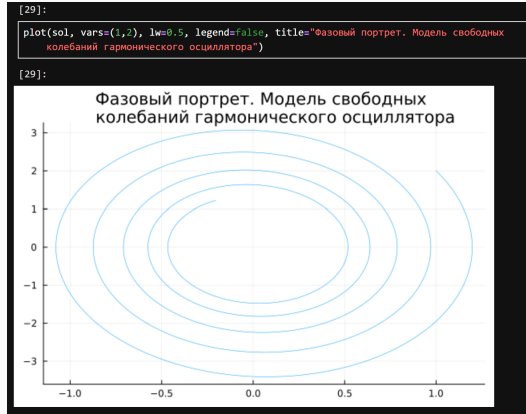


Рис. 33: Задание для самостоятельного выполнения №8

Задания для самостоятельного выполнения

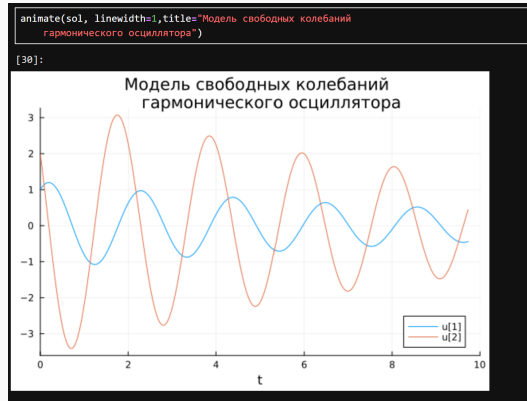


Рис. 34: Задание для самостоятельного выполнения №8

В результате выполнения работы были освоены пакеты для решения задач в непрерывном и дискретном времени.